ESP32 SMART DUSTBIN

Complete Prototype Building Guide

Ultrasonic fill-level monitoring | Wi-Fi | Python Flask server | Browser dashboard

Scope: Build a working monitoring prototype without an RGB LED. Optional automated voice-call alerts are a later integration.

Safety first: Do not wire the ultrasonic Echo output directly to an ESP32 GPIO if it may be 5 V. Your reported 1 ohm and 2.2 ohm resistors are not suitable for the required divider.

System part	What it does
AJ-SR04M waterproof sensor	Measures distance from the bin lid to waste
ESP32	Calculates fill percentage and sends data over Wi-Fi
Flask application	Accepts HTTP updates and serves a browser dashboard
Dashboard	Displays fill percentage, status and latest measurement

1. Components and prerequisites

Item	Status / note
ESP32-WROOM-32 development board	Available; previously connected to Wi-Fi
AJ-SR04M waterproof ultrasonic module + probe	Available; verify module pinout and trigger/echo mode
USB cable and jumper wires	Available
1 ohm and 2.2 ohm resistors	Available but DO NOT use as Echo divider
1 kohm and 2 kohm resistors	Obtain these, or an appropriate equivalent divider
Computer with Arduino IDE and Python 3	Required
Dustbin or cardboard model	Required for mounting and calibration
RGB LED / servo motor	Not required for this monitoring prototype

2. Electrical safety and wiring

Disconnect USB power before wiring. The ESP32 GPIO is a 3.3 V interface. Some AJ-SR04M variants output approximately 5 V on Echo. Check the markings and documentation for your exact module. The following wiring assumes a compatible trigger/echo mode and a 5 V supply.

Sensor pin	ESP32 / circuit connection
5V or VCC	5V / VIN pin (with ESP32 powered by USB)
GND	GND; shared ground
TRIG	GPIO 5
ECHO	Through 1 kohm series resistor to divider junction; junction to GPIO 18
Divider junction	Through 2 kohm resistor to GND

Correct divider: ECHO (5 V) -> 1 kohm -> junction -> GPIO 18; junction -> 2 kohm -> GND. Expected junction voltage: 5 x 2000 / (1000 + 2000) = 3.33 V. If possible, choose a divider that outputs 3.0-3.3 V for extra margin.

Never substitute 1 ohm / 2.2 ohm: Those values would draw excessive current. Also, 1 kohm + 2.2 kohm gives about 3.44 V at a 5 V input, so the 1 kohm + 2 kohm pair is preferred.

3. Mechanical installation

Attach the waterproof probe under the lid facing down. Keep it clear of sidewalls and hanging bags. Keep the controller board and ESP32 dry and mechanically supported. Measure the actual distance from probe to the empty-bin bottom and the distance representing the desired full threshold. Some waterproof ultrasonic modules have a near-field blind zone: confirm that the full threshold is beyond the module's minimum measurable distance.

4. Arduino IDE setup

Install the ESP32 board package in Arduino IDE. Select the appropriate ESP32 Dev Module and COM port. Open Serial Monitor at 115200 baud. Replace the Wi-Fi name, password, PC IP address and calibration distances in the code below. ESP32 and PC must be on a network that allows device-to-device communication.

5. ESP32 firmware

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <math.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* serverURL = "http://YOUR_PC_IP:5000/api/update";

#define TRIG_PIN 5
#define ECHO_PIN 18
const float EMPTY_DISTANCE_CM = 40.0; // calibrate
const float FULL_DISTANCE_CM  = 5.0;  // calibrate

float measureDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return -1;
  return duration * 0.0343f / 2.0f;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  digitalWrite(TRIG_PIN, LOW);
  WiFi.begin(ssid, password);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts++ < 30) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(WiFi.status() == WL_CONNECTED ?
                 "\nWi-Fi connected" : "\nWi-Fi failed");
  if (WiFi.status() == WL_CONNECTED) Serial.println(WiFi.localIP());
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.reconnect();
    delay(5000);
    return;
  }
  float distance = measureDistance();
  if (distance < 0) {
    Serial.println("Sensor timeout");
    delay(3000);
    return;
  }
  int fill = constrain((int)roundf(
    100.0f * (EMPTY_DISTANCE_CM - distance) /
    (EMPTY_DISTANCE_CM - FULL_DISTANCE_CM)), 0, 100);
  Serial.printf("Distance: %.1f cm | Fill: %d%%\n", distance, fill);
  HTTPClient http;
  http.begin(serverURL);
  http.addHeader("Content-Type", "application/json");
  http.setTimeout(5000);
  String payload = "{\"distance\":" + String(distance, 1) +
                   ",\"fill\":" + String(fill) + "}";
  int response = http.POST(payload);
  Serial.printf("Server response: %d\n", response);
  http.end();
  delay(3000);
}
```

6. Python Flask backend and dashboard

On your PC create a folder named smart-dustbin. In a terminal run: python -m pip install flask. Save the following as app.py. This starter backend keeps only the latest reading in memory; data resets when the server restarts.

```
from flask import Flask, request, jsonify, render_template_string
from datetime import datetime
```

```
app = Flask(__name__)
latest = {"distance": None, "fill": 0, "status": "WAITING",
          "updated": "No readings yet"}

HTML = """<!doctype html><html><head>
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Smart Dustbin</title>
<style>
body{font-family:Arial;background:#101827;color:white;
 text-align:center;padding:35px}
.card{max-width:480px;margin:auto;background:#1e293b;
 border-radius:16px;padding:28px}
.track{height:24px;background:#334155;border-radius:12px;
 overflow:hidden}
#bar{height:100%;width:0;background:#22c55e}
#fill{font-size:52px;font-weight:bold}
</style></head><body><div class="card">
<h1>Smart Dustbin Monitor</h1><div id="fill">0%</div>
<div class="track"><div id="bar"></div></div>
<h2 id="status">WAITING</h2><p id="distance">Distance: --</p>
<p id="updated">Waiting for sensor...</p></div>
<script>
async function refresh(){
 try{
  let r=await fetch('/api/status');let d=await r.json();
  document.getElementById('fill').textContent=d.fill+'%';
  document.getElementById('status').textContent=d.status;
  document.getElementById('distance').textContent=
    'Distance: '+(d.distance??'--')+' cm';
  document.getElementById('updated').textContent=
    'Updated: '+d.updated;
  let bar=document.getElementById('bar');
  bar.style.width=d.fill+'%';
  bar.style.background=d.fill>=80?'#ef4444':
    d.fill>=50?'#f59e0b':'#22c55e';
 }catch(e){document.getElementById('status').textContent=
    'SERVER OFFLINE';}
}
setInterval(refresh,2000);refresh();
</script></body></html>"""

@app.get('/')
def dashboard():
    return render_template_string(HTML)

@app.get('/api/status')
def status():
    return jsonify(latest)

@app.post('/api/update')
def update():
    data = request.get_json(silent=True) or {}
    try:
        distance = float(data['distance'])
        fill = int(data['fill'])
    except (KeyError, ValueError, TypeError):
        return jsonify(error='Invalid reading'), 400
    if distance < 0 or not 0 <= fill <= 100:
        return jsonify(error='Out of range'), 400
    latest.update(distance=round(distance,1), fill=fill,
        status=('FULL - COLLECT NOW' if fill>=80 else
                'ALMOST FULL' if fill>=50 else 'AVAILABLE'),
        updated=datetime.now().strftime('%H:%M:%S'))
    print('Reading:', latest)
    return jsonify(success=True)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

7. Run the system

1. Start the Flask server with python app.py. Open http://localhost:5000 on the PC.

2. On Windows, run ipconfig to find the PC Wi-Fi IPv4 address. Enter it into the ESP32 serverURL (for example http://10.160.27.50:5000/api/update - this address is illustrative only).

3. Permit inbound TCP port 5000 on your trusted private network if the firewall blocks the ESP32. Some public or guest networks isolate clients; use a private router/hotspot that permits local connections.

4. After confirming wiring and sensor mode, power the board, upload firmware and inspect Serial Monitor. Successful posts typically return HTTP 200. The dashboard refreshes every two seconds.

8. Calibration and acceptance testing

Measure the sensor-to-bottom distance with the bin empty; enter it as EMPTY_DISTANCE_CM. Measure the sensor-to-garbage distance at your intended full condition; enter it as FULL_DISTANCE_CM. Keep EMPTY_DISTANCE_CM greater than FULL_DISTANCE_CM. Formula: fill % = 100 x (empty distance - measured distance) / (empty distance - full distance), clipped to 0-100.

Test	Expected result
Wi-Fi	Connected; valid IP and usable signal
Empty bin	Reading near 0%; status AVAILABLE
Part-filled bin	Percentage rises as the measured distance falls
50%-79%	Status ALMOST FULL
80%-100%	Status FULL - COLLECT NOW
Flask server stopped	ESP32 shows failed HTTP request; page cannot refresh
Sensor disconnected / no echo	Sensor timeout; no false fill update

9. Troubleshooting

Symptom	Check
No sensor readings	AJ-SR04M operating mode, probe connector, power, TRIG/ECHO pins and divider
Readings always zero / too high	Recalibrate empty/full distances and confirm sensor blind zone
ESP32 restarts	USB power quality, shorts, and incorrect low-value resistors
HTTP response negative	PC address, same LAN, server running, firewall and client isolation
Dashboard stuck on WAITING	Check ESP32 serial POST response and Flask terminal logs
Unstable readings	Mounting, reflections, moving waste; add median filtering later

10. Future enhancements

Once the monitoring prototype works: add persistent storage, multiple dustbin IDs, map positions, a history chart, secure device authentication, and a call-provider API. A voice-call workflow should include consent/authorized recipients, rate limits and deduplication to avoid repeated calls. An automatic lid requires additional hardware such as a servo motor and a suitable power supply; it is not part of the current build.

Final checklist: correct resistors obtained; sensor mode verified; all grounds shared; sensor dry and securely mounted; calibration measured; Flask running; PC IP configured; dashboard updates from real sensor data.